EXP 10. A* Algorithm

Aim

To find a least-cost path from a start node to a goal node using the A* search algorithm.

Algorithm

1. Represent the graph with edge costs and define a heuristic estimate h(n) for each node.
2. Insert the start node into a priority queue with priority f(n) = g(n) + h(n).
3. Remove the node with the smallest f value.
4. If it is the goal, return the path and cost.
5. Otherwise, calculate new costs for its neighbours and update any better paths.
6. Repeat until the goal is found or the queue becomes empty.

Program

AI.py - C:/Users/Barath/AppData/Local/Programs/Python/Python313/AI.py (3.13.15)

File Edit Format Run Options Window Help

```
import heapq

graph = {
    "A": [("B", 1), ("C", 3)],
    "B": [("D", 3), ("E", 1)],
    "C": [("F", 5)],
    "D": [],
    "E": [("F", 2)],
    "F": []
}
h = {"A": 6, "B": 4, "C": 5, "D": 3, "E": 2, "F": 0}

def astar(start, goal):
    pq = [(h[start], 0, start, [start])]
    best = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g

        if g != best[node]:
            continue

        for nxt, cost in graph[node]:
            new_g = g + cost
            if new_g < best.get(nxt, float("inf")):
                best[nxt] = new_g
                heapq.heappush(
                    pq, (new_g + h[nxt], new_g, nxt, path + [nxt])
                )
    return None, float("inf")

path, cost = astar("A", "F")
print("Path:", " -> ".join(path))
print("Cost:", cost)
```

Output

```
================================================
Path: A -> B -> E -> F
Cost: 4
>
```

Result

Thus, the A* algorithm found the path A → B → E → F with total cost 4.